https://github.com/aymanketat/mai580-agent

MAI 580 — Milestone 1 (Stages 1–5): Progress Note

Ayman Ketat · Use Case B: Research Assistant (questions about machine-learning papers) · 24 October 2026

Submitted version: tag `stage-05` **= commit** `e996a70` (archive `milestone1.zip`; its test suite: 15 passed). Later commits on `main` (up to `413d918`: review fixes to `check_citation`, citation-count caching, Retry-After handling, 65 tests) are improvements made after `stage-05` and are not in the archive.

Model `gpt-4.1-mini`, temperature 0; langgraph 1.2.12, langchain 1.4.2; data: 23 arXiv records in `data/papers.json`, Semantic Scholar counts (live).

Stage 1 — one LLM node (`stage-01`)

Graph: START → model → END (one `State` field `messages` with the `add_messages` reducer; node `call_model`).

Five questions (one fresh run each; ground truth `data/papers.json`):

#	question	needs	result
1	How does LoRA differ from full fine-tuning, when to use it?	general	correct
2	Who wrote the ResNet paper, what year?	general	correct (He, Zhang, Ren, Sun; 2015)
3	Citation count of *Attention Is All You Need* now?	live data	honest refusal
4	Which papers in **my saved catalogue** are about LLM agents, how many in total?	**private data**	honest refusal ("no access to your catalogue")
5	Exact arXiv id and latest version of *Auto-Encoding Variational Bayes*?	rare metadata	**confident invention**: id right, "v10" (catalogue: v11)

Memory test (two separate runs): "My name is Sara." → "Hello Sara!"; then "What is my name?" → "I don't have access to your personal information". Each `invoke` starts from an empty state (2 messages each run).

Reflections. (1) A node returns an *update*; the `add_messages` reducer appends it, so the node need not know the history and several nodes can add messages without overwriting each other. (2) Without a reducer the returned list *replaces* `messages`: after Stage 1 only the reply would remain; nodes that write only `analysis`/`draft` would not touch it. (3) Wrong: Q5's version (v10 vs v11) and, on the Stage 2 set, "Llama 2 has 17 authors" (68) and a wrong title for 1908.10084 — long-tail metadata that training text holds rarely; the model cannot look anything up, and temperature 0 only makes the guess repeatable.

Stage 2 — sequential workflow (`stage-02`)

Graph: START → analyze → generate → review → END; fields `analysis`, `draft`. **Same 10 questions, both graphs** (3 blind LLM graders, median 1–5; tokens per LLM call):

system	latency	tokens in / out	quality	factual errors
Stage 1 single node	34.7 s	307 / 4 330	3.9	2 answers
Stage 2 workflow	66.4 s	8 257 / 7 998	3.4	2 answers (+3 stale/unsupported additions)

Not worth 3× the calls on average: 1.9× latency, 27× input tokens, lower quality. It helped on the fake-reference check and long trade-off answers (concise, cites papers); it hurt on short facts: the review node hedged correct answers and certified "13 authors" for Llama 2.

Reflections. (1) `review` reads only `draft` and writes `messages`; `analysis` and `draft` stay in the state as a trace. (2) Wrong path: "What year did the BERT paper come out?" — three calls and 274 input tokens for a one-word answer the single node got in 0.7 s; equally, a citation-count question that no analysis can answer without a tool.

Stage 3 — conditional routing (`stage-03`)

Graph: `classify` (structured output over `paper_lookup / literature_search / citation / concept / other`) → one answer node per intent → `END`. **20 labelled requests** (5/5/4/4/2):

descriptions	accuracy	confusion
v1: "citation = produce a BibTeX/APA citation, or verify/correct a reference"	18/20 = 90%	2 × citation → paper_lookup (both were *reference checks* mentioning authors/year); all other cells on the diagonal
v2: names the cue "the student is working on a reference or bibliography entry … even if the question mentions authors or a year"	20/20 = 100%	diagonal; tuned on the same 20 items, so an upper bound

Reflections. (1) `with_structured_output` sends an enum schema, so the model can only return one of five labels — never free text that would `KeyError` in the routing map. (2) Two-intent requests ("look up ReAct and give me its BibTeX", "find papers on X and say which is most cited"): a single-label router must pick the branch that can finish the whole request; the better answer is Stage 4–5, routing anything that touches paper data to one tool-using branch.

Stage 4 — tool use, one round (`stage-04`)

Graph: router + agent → tools → final_answer; tools `search_papers`, `get_paper`, `format_citation` (errors returned as data). **Stage 1 data questions repeated with tools:**

question	tool calls	result
Catalogue papers about LLM agents / total	`search_papers` ×3	7 matches listed (incl. lexical noise); total not exposed by any tool
VAE paper id and latest version	`search_papers`	id 1312.6114 correct; version "not retrieved" (card lacks it; no 2nd round)
Llama 2 author count	`search papers`	**68, correct** (Stage 1: 17)
Is "Gated Memory Compression…", arXiv 1908.10084, real?	`get_paper`	**correct**: it is Sentence-BERT (Stage 1 named a wrong paper)
Citation count of *Attention…*	`search papers`	honest "could not retrieve" (no such tool yet)

0 inventions (Stage 1: 3). **Two-round request** ("find the paper that introduced Sentence-BERT and give me its BibTeX"): messages = Human → AI(`search_papers`) → ToolMessage → AI(answer). `final_answer` has no tools, so after seeing the id it could not call `format_citation` and **wrote the BibTeX itself** from the card and memory (likewise an invalid "Shinn, N., et al." APA for Reflexion): untraceable to any tool result.

Reflections. (1) A deliberately bad docstring ("Does something with a paper.") did not change selection (3/3), nor did renaming the tool `tool_c`: the model reads the whole schema (the `style` enum `apa|bibtex|ieee`) and eliminates against two tools that clearly do not cite; docstrings matter when tools overlap. (2) Tool tests without the LLM are deterministic, free and isolate tool bugs — a wrong BibTeX key (`devlin2018paper`) was caught by `format_citation.invoke(...)`. (3) The loop is not closed: tools → final_answer → END gives the model one decision, so any second step that depends on the first result is guessed.

Stage 5 — the agent loop (`stage-05`)

Change: tools → agent (no `final_answer`); `recursion_limit = 12` with a fallback message (verified: limit 3 raises `GraphRecursionError`, 12 finishes; this install's default is 10 007); tools `get_citation_count`, `check_citation`. **10 multi-step requests + 1 control** (3 blind graders; one rubric applied to both; evidence re-assessed after review):

multi-step requests 1–10	Stage 4: 1 round, 3 tools	Stage 5: loop, 5 tools
verified success (all parts, correct, citations/counts traceable) · content-only	**0/10** · 3/10	**5/10** (+1 unverified) · 5/10
factual errors · citations without tool provenance	2 · 4	0 · 0
tool calls · input tokens · latency	13 · 11 980 · 37.2 s	36 · 44 291 · 50.6 s
mean score (recorded medians) · control request 11	2.10 · 1/1	4.00 · 1/1

Stage 5's four misses were Semantic Scholar rate limits, reported honestly; Q5's "most cited" is unverified (tool results were not logged).

5 of 10 requests need a count only Stage 5 can fetch; on the 5 solvable with Stage 4's tools but needing two dependent calls: 0/5 vs 5/5 — consistent with a benefit from the loop, not a controlled isolation.

Unnecessary call: Q9 — `check_citation` already returned Reflexion's `arxiv_id`, yet the agent ran `search_papers("Reflexion … Yao 2023")` before `get_citation_count`; in Q6 the same tool's id was used directly. The prompt says "find the arxiv_id first" and the model applies the rule even when the id is already in a ToolMessage (same pattern: re-searching an id after `get_paper` in Q4/Q10). Graders counted 8 unnecessary calls in 11 requests.

Diagrams. Hand-built: START → classify; `classify ⇢ agent | answer_concept | answer_general`; `agent ⇢ tools` (tool_calls) or `⇢ END`; tools → agent. `create_agent(model, tools=TOOLS, system_prompt=…)`: START → model; `model ⇢ tools | END`; `tools ⇢ model`. Same loop; the prebuilt names the LLM node `model`, has no router or direct branches, draws tools → model as conditional (it can end after a `return_direct` tool), and prepends the system prompt at call time.

Reflections. (1) The single edge tools → agent (replacing tools → final_answer) makes the deciding node the one that sees its results. (2) When the budget runs out: stop, say plainly what was found and what was not (the collected ToolMessages are evidence), log it for Stage 12/13; do not silently retry with a bigger budget — the loop usually ran out because it was repeating itself. (3) `recursion_limit` is both a cost control (each round re-sends the history: 3.7× input tokens here) and a safety control (bounds a misbehaving or injected loop), but it counts steps, not money or risk: Stage 12 adds cost, Stage 14 per-tool authorisation.

Contribution and AI disclosure. I selected Use Case B and am working through the code and results. I used claude assisted with fixing code errors , testing and formalize documentation , it drafted the stage code from the tutorial starter, the experiment scripts and question sets.